



Society of Petroleum Engineers

SPE-229648-MS

Smart Inspection for Corrosion and Equipment Monitoring Using Robotics with Cloud-Based Computer Vision

D. Shekhawat, SLB, Crawley, United Kingdom; J. Barua, SLB, Aberdeen, United Kingdom; K. Bhatia, SLB, Crawley, United Kingdom

Copyright 2025, Society of Petroleum Engineers DOI [10.2118/229648-MS](https://doi.org/10.2118/229648-MS)

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Abstract

The objective of this work is to develop and operationalize a smart inspection system that uses cloud-hosted computer vision models and robotic inputs to detect and classify corrosion, read equipment status, and improve asset integrity at surface facilities. The solution aims to enhance operational efficiency and reduce manual inspection workloads at offshore facilities. Methods, Procedures, Process: The system leverages deep learning models (ResNet for corrosion segmentation and YOLO for equipment detection) trained on historical inspection and open-source datasets. These models are deployed on a cloud-native environment for scalable inference. A robotic or mobile camera captures facility images, which are then processed via a multi-threaded data workflow. Detection outputs, metadata, and inspection results are stored using a standardized schema. A python-based user interface provides end users with interactive visualizations, filtering, and measurement correction capabilities. The complete system supports gauge reading, valve state detection, and corrosion classification with a quantifiable corrosion score.

The system successfully detects corrosion and equipment states from visual inputs across a variety of surface facility environments. For corrosion detection, the model generates pixel-level segmentation masks, categorizes corrosion severity, and outputs a corrosion score on a scale of 0-100. For equipment monitoring, the system identifies valve states (open/close) with bounding boxes and confidence scores, while also calculating gauge readings using geometric anglebased interpretation. Multithreaded execution reduced inference latency by ~40%, and cloud integration enabled processing scalability without additional local infrastructure. Cloud-based architecture allows users to upload new images, run inference workflows, and view results directly through a web interface. Data is automatically structured, stored, and linked via metadata, supporting historical traceability and audit. Despite training on limited datasets, the models demonstrated decent generalization across unseen images, with model accuracy expected to improve as more labeled inspection data becomes available. This approach transforms conventional inspection by offering consistent, fast, and intelligent analysis, reducing the need for manual interpretation and helping teams focus on high-risk anomalies. It represents a step forward in integrating computer vision and cloud robotics into mainstream asset integrity workflows

This paper introduces a practical and scalable application of AI for surface facility inspection by combining robotics, cloud computing, and computer vision. Unlike conventional methods, it offers real-time feedback, multithreaded processing, and a robust cloud-hosted workflow, making it easier to deploy and scale across operations. It contributes to the growing use of AI in field-level operational advancements and digital asset integrity management

Introduction

Maintaining the integrity of offshore surface facilities is a cornerstone of safe and efficient oil and gas production. These environments are often complex, geographically remote, and subject to harsh operating conditions such as high humidity, saltwater exposure, and temperature fluctuations. Equipment within these facilities, such as valves, gauges, and pipelines, is prone to corrosion, wear, and malfunction over time, necessitating frequent inspections. Traditional inspection methods, which rely heavily on manual data collection and visual interpretation by field personnel, are increasingly viewed as insufficient due to safety risks, operational delays, and inconsistency in data quality.

To address these challenges, the industry is seeing the growing adoption of robotics for visual inspection. Mobile robots, including wheeled crawlers, aerial drones, and quadruped platforms, are being deployed to capture images and videos of facility equipment, particularly in environments that may be hazardous, confined, or otherwise difficult to access. These robotic systems offer advantages such as consistent coverage, reduced human exposure, and the ability to perform inspections more frequently or in hard-to-reach locations.

However, despite the growing presence of robotic platforms in offshore operations, the majority of inspection workflows remain heavily manual, particularly in the interpretation and analysis stages. While drones and mobile robots are increasingly used to capture high-resolution images or video of facility equipment, the downstream processing of this visual data is still largely dependent on human experts. Engineers or inspectors are often required to review the imagery, identify potential anomalies, annotate areas of concern, and document findings using spreadsheets or written reports. This manual post-processing introduces subjectivity, slows down the overall inspection cycle, and increases the risk of missed defects or delayed interventions. As offshore assets age and the volume of inspection data grows, this approach becomes increasingly inefficient and unsustainable.

In recent years, artificial intelligence (AI), and specifically deep learning-based computer vision, has emerged as a powerful enabler for automating visual inspection. Convolutional neural networks (CNNs) such as ResNet, and object detection models like YOLO (You Only Look Once), have demonstrated strong performance in industrial applications including image segmentation, object localization, and equipment classification. These models can be trained to identify corrosion regions, detect valve positions, read analog gauge values, and estimate the severity of equipment degradation directly from image inputs. By embedding such models into inspection workflows, operators can significantly improve the consistency, speed, and accuracy of their integrity assessments.

Yet, despite the potential of AI technologies, their deployment in field operations remains limited. In many cases, AI-based inspection tools are implemented in isolated environments, often running on local machines, confined to lab setups, or disconnected from enterprise systems. These siloed approaches prevent real-time insight delivery, restrict scalability across assets, and hinder integration into broader digital operations. As a result, even where AI is introduced, its impact remains constrained by the lack of end-to-end system integration.

To overcome these limitations, there is an increasing need for a unified, cloud-native architecture that seamlessly combines robotic data acquisition, intelligent image analysis, and intuitive user interfaces for field personnel and integrity teams. A centralized system enables scalable inference, automated result

structuring, and secure access to inspection outputs from any location. This transformation supports the industry's broader digitalization goals, enhancing operational efficiency, safety, and decision-making.

This paper presents a practical implementation of such a system: a smart inspection solution that leverages robotic image capture, deep learning models, and cloud infrastructure to automate corrosion detection, valve state recognition, and analog gauge reading. Built for scalability and operational integration, the system has been deployed and validated in North Sea operator environments. It represents a significant step toward autonomous inspection workflows, reducing reliance on manual interpretation, and improving the reliability and consistency of surface facility monitoring.

Methodology

The design of the smart inspection system was guided by a clear objective: to build a scalable, production-grade solution capable of handling large volumes of robotic image data, supporting multiple inspection use cases, and meeting the operational needs of diverse offshore clients. From the outset, authors prioritized performance, modularity, and reusability, ensuring that the system could be deployed and extended across different facilities without major redevelopment.

To achieve this, authors implemented a cloud-based Operational Data Foundation (ODF) [Fig1] as the central backbone of the architecture[1]. This decision was instrumental in addressing the fundamental challenges associated with robotic inspections: data volume, asset contextualization, real-time processing, and multi-client adaptability. Robotic surveys generated thousands of high-resolution images per inspection run, each tied to specific components such as valves, gauges, or piping segments. Authors configured the ODF to ingest this unstructured data at scale and automatically associate it with structured asset hierarchies, metadata, and inspection schedules. Each image was tagged with identifiers, equipment type, facility code, timestamp, and inspection type, allowing for efficient retrieval and correlation with field operations.

Using the ODF, authors also designed a dedicated Robotics Data Model (RDM) to support inspection-specific workflows. The RDM was implemented using GraphQL-based data modelling, which enabled visual design and iterative feedback from robotics data experts during development. This collaborative modelling approach allowed us to define key relationships between images, survey metadata, asset tags, and model outputs. Metadata such as image capture time, camera orientation, inspection route, and facility segment were automatically extracted from relevant files, enabling automated contextualization of incoming surveys. This eliminated the need for manual tagging and ensured that newly uploaded datasets could be processed and classified with minimal human intervention.

For AI model execution, authors developed and deployed detection functions directly within the ODF environment. This included image pre-processing pipelines, deep learning inference logic, and post-processing steps. The use of multithreaded, serverless functions allowed us to execute model inference in parallel, reducing latency by approximately 40% during batch runs. This ensured the system could handle high-throughput image workflows without bottlenecks or manual intervention. Authors structured all results, whether bounding boxes, segmentation masks, or severity scores, using a standardized schema. This enabled seamless integration with downstream dashboards and ensured that every inference output was traceable back to its input image, model version, and execution timestamp. For visualization and human-in-the-loop review, authors deployed a python-based UI that allowed users to upload images, trigger workflows, and interactively analyze results.

The architecture was modular by design. Authors decoupled the vision models, workflow orchestration, and UI layers to enable independent updates and maintenance. This proved critical when adapting the system across multiple client environments, each with different asset classes, inspection priorities, and data quality constraints. The same backend was used to support corrosion detection, valve state recognition, and analog gauge reading, simply by switching the model modules and adjusting the processing logic.

By leveraging the ODF and designing a fit-for-purpose RDM, authors delivered a system that was not only technically sound but also operationally aligned with field realities. It supported scalability, reduced manual workloads, and enabled faster, more consistent inspection outcomes across multiple North Sea operator sites [2].

Robotic Data Ingestion and Contextualization

To streamline the transfer and structuring of robotic survey data into the Operational Data Foundation (ODF), authors developed a dedicated ODF Extractor Utility. This component was designed to operate close to the data source, typically hosted on-site or within a secured cloud connector, and was capable of monitoring predefined target locations, such as structured folders in SharePoint, for the arrival of new survey data. Once a new survey was detected, the extractor performed schema and format validation before automatically ingesting the files into the ODF.

In addition to automated ingestion, the system also allows a manual upload interface[Fig1] via the system's web UI. This allowed inspection engineers to perform ad-hoc analysis using non-robotic data sources, such as handheld camera images or legacy inspection snapshots. Both robotic and manual data followed the same downstream workflow, giving engineers consistent experience while preserving flexibility. This hybrid upload model ensured that the system could accommodate structured survey campaigns as well as rapid-response inspections without introducing manual bottlenecks.

Once the survey data entered the system, an intelligent contextualization workflow was triggered. Metadata, such as Survey ID, capture date, GPS coordinates, altitude, camera orientation, and equipment markers, was automatically extracted from image headers (EXIF), drone logs, filenames, and directory structure. Where structured metadata was incomplete or ambiguous, authors implemented rule-based logic and heuristics to infer missing associations. This allowed us to programmatically assign each image to its corresponding equipment type, facility zone, or inspection route. Contextualization outputs were written directly into the RDM, ensuring each image was properly linked to its operational context.

To support high-volume survey ingestion, authors enabled parallel processing of files across multiple threads. The ODF Extractor Utility was designed to batch and dispatch file groups concurrently, allowing for rapid preprocessing, metadata extraction, and data model initialization. This approach significantly reduced processing time for large surveys, achieving throughput improvements of 2x to 3x compared to sequential workflows. The architecture supported scale-out deployments, meaning that multiple extractor instances could run in parallel across different facilities or asset groups, each feeding data into a centralized ODF backend.

The ingestion and contextualization pipeline were designed to be schema-aware, allowing automatic alignment with the underlying RDM without requiring manual data wrangling. This allowed newly uploaded surveys to be immediately available for inspection analytics, with consistent metadata structure across survey runs, facilities, and client environments. In effect, authors created a self-aligning ingestion framework that prepared visual data for downstream analytics with minimal human intervention, ensuring speed, accuracy, and operational traceability.

By combining automation, configurability, and scalable execution, this ingestion layer served as the digital front door for the inspection system, handling complexity behind the scenes while enabling engineers to focus on high-value analysis and decision-making.

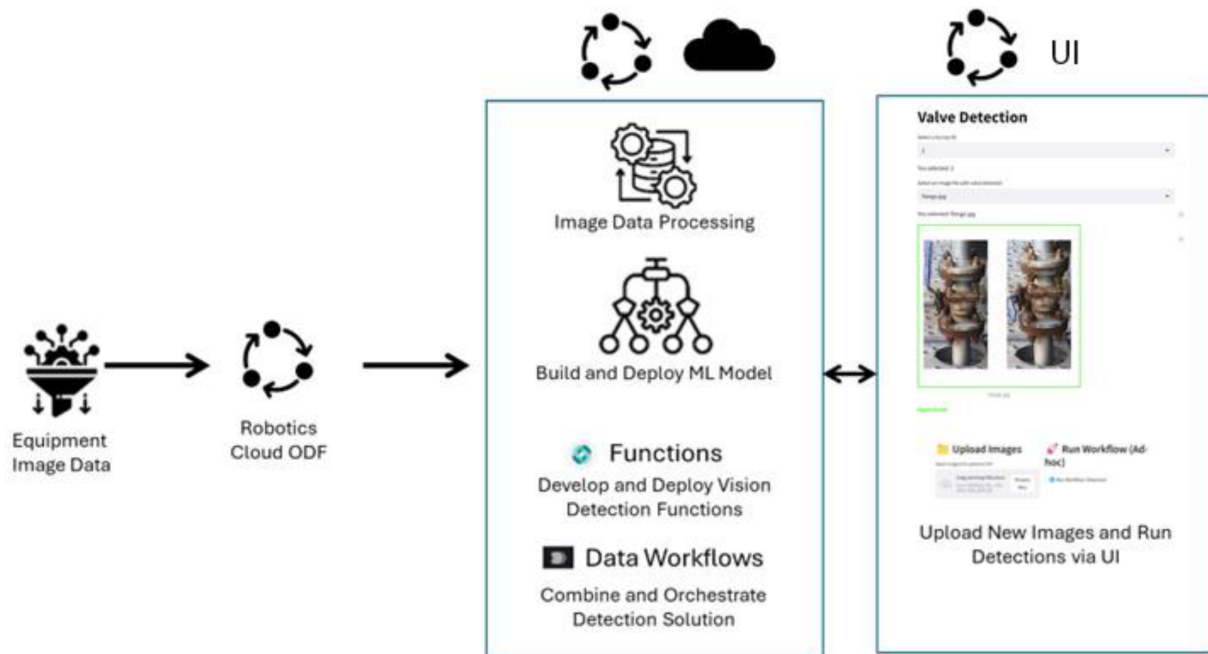


Figure 1—ODF Architecture

Vision Analytics

The Vision Analytics module is the core inference engine of the smart inspection system, responsible for converting structured image data into machine-interpretable insights using deep learning-based computer vision techniques. This layer leverages supervised learning architectures trained on annotated datasets to identify, localize, and quantify various inspection targets from visual inputs captured during robotic or manual surveys.

Two primary computer vision paradigms were employed to support a range of visual inspection tasks: image classification and image segmentation. These paradigms were implemented using state-of-the-art convolutional neural network (CNN) architectures: YOLO (You Only Look Once) for object detection and classification, and ResNet-based encoder-decoder networks for semantic segmentation [3] [4].

Image Classification and Object Detection. Image classification refers to assigning one or more categorical labels to an image or sub-region. When extended to object detection, the task becomes a combination of class probability prediction and bounding box regression for every object instance within an image.

Authors implemented object detection using the YOLO architecture, which performs detection in a single forward pass across multi-scale feature maps. The network outputs a dense prediction tensor containing class probabilities, bounding box offsets, and confidence scores. Non-Maximum Suppression (NMS) was applied post-inference to eliminate redundant detections.

YOLO Model Training Methodology. To train the detection model, authors constructed a hybrid dataset that combined:

- **Operator-sourced inspection imagery** captured via robotic platforms from actual offshore assets, annotated with component types (e.g., valve, gauge, panel).
- **Public and synthetic datasets** to introduce diversity in lighting, weathering, camera angles, and equipment appearance.

Each image was labelled using bounding boxes in YOLO format (class_idx_center_x_center_y_center_x_center_y normalized to image dimensions). Labelling was performed using an internal annotation interface, with quality assurance by domain experts to ensure annotation accuracy and class consistency.

To enhance generalization, authors applied extensive data augmentation during training:

- Random scale and crop
- Horizontal flipping
- Hue/saturation shifts
- Motion blur and Gaussian noise
- Mosaic and cutout augmentations (to simulate cluttered environments)

Training was conducted using PyTorch with stochastic gradient descent (SGD) optimizer, cosine annealing learning rate schedules, and mixed-precision training for efficiency. Model performance was evaluated using mean Average Precision (mAP@0.5) and IoU(intersection over union) thresholds, tuned across multiple validation folds derived from field data distributions.

Image Segmentation. Image segmentation assigns class labels to individual pixels in an image, producing a dense output tensor for per-pixel classification. This is especially suitable for quantifying surface-based phenomena such as rust patches, paint delamination, or chemical staining.

Authors used a ResNet34 encoder–decoder segmentation network implemented via a U-Net style architecture, incorporating skip connections and bilinear up sampling for spatial reconstruction.

Segmentation Model Training Methodology. The segmentation model was trained using a **curated dataset of corrosion imagery** consisting of:

- Field images from customer inspections labelled at the pixel level using manual labelling utilities.
- Public datasets of rust, pitting, and oxidation on metallic surfaces under varied lighting and backgrounds.

Each training image was paired with a binary mask (corrosion vs. non-corrosion), saved in rasterized format. Masks were quality-checked and refined to ensure clear edge boundaries and label fidelity.

To address class imbalance, common in datasets where corrosion affects only a small portion of the image, authors used **Dice Loss** combined with **Focal Loss**. Data augmentation strategies included:

- Brightness and contrast jitter
- Random affine transformations
- Perspective warping
- Patch-based training to ensure adequate coverage of both foreground and background pixels

Training employed the Adam optimizer, batch normalization, and a cyclical learning rate strategy. Validation metrics included mean Intersection over Union (mIoU), Dice coefficient, and False Positive Rate (FPR). Model checkpointing and early stopping were used to retain the best-performing weights and prevent overfitting.

By training on both field-acquired and external data, authors achieved models capable of robust performance under variable real-world inspection conditions.

This dual-model vision analytics framework, grounded in consistent training methodology, data diversity, and deep learning best practices, enabled accurate and scalable interpretation of robotic inspection data across multiple industrial contexts.

Corrosion Detection and Severity Scoring

Corrosion is a progressive and visually irregular degradation phenomenon common in offshore surface facilities, driven by prolonged exposure to saltwater, humidity, and chemical contaminants. It can manifest as staining, blistering, scabbing, or flaking, and if left undetected, may escalate to structural failures, leaks, or costly interventions. Even with the adoption of robotic imaging tools, the interpretation of corrosion imagery remains largely manual, introducing subjectivity and delays. To overcome these limitations, a computer vision model was developed to detect, classify, and quantify corrosion from visual inputs captured during inspections.

Authors considered both object detection and semantic segmentation architectures. While YOLO-based object detection offers fast inference and effective localization for geometric objects, it was deemed inadequate for corrosion detection. Corrosion spreads in diffuse, irregular patterns, lacks clear boundaries, and often includes overlapping classes within a single region. These characteristics necessitate pixel-wise classification rather than bounding box approximation. Therefore, semantic segmentation was selected as the more appropriate technique for this application.

The segmentation model was based on a U-Net architecture with a ResNet34 encoder, chosen for its balance between feature extraction capability and computational efficiency. The model input consisted of RGB images resized to 256×256 pixels. The output was a pixel-level mask where each pixel was classified into one of four corrosion classes:

1. Class 0 (no corrosion),
2. Class 1 (early corrosion staining)
3. Class 2 (blistering or scabbing)
4. Class 3 (flaking)

The colour coding followed a consistent convention for interpretability in downstream visualizations. The dataset used for training and evaluation comprised 1,000 labelled images, combining field data from robotic inspections and corrosion samples from public datasets. The data was split in a 90/10 ratio for training and testing respectively.

Training was conducted in PyTorch using the Adam optimizer with cosine annealing learning rate scheduling and mixed-precision support. A composite loss function combining Dice Loss and Focal Loss was employed to manage class imbalance and focus on difficult-to-classify pixels. Data augmentation included affine transformations, brightness and contrast adjustments, and patch-based sampling to ensure representative exposure to minority classes and varied corrosion morphologies. The segmentation masks were verified for consistency and labelled at pixel level using internal tools.

To translate the segmentation output into a practical engineering metric, authors designed a corrosion severity scoring algorithm. This algorithm calculates the total area affected by corrosion and applies class-specific weights to represent severity. Specifically, weights of 5, 15, and 80 were assigned to Classes 1, 2, and 3 respectively, while Class 0 (no corrosion) was excluded from scoring. The weighted sum of pixels was normalized against the maximum possible corrosion weight to produce a scalar severity score between 0 and 100. This Corrosion Severity Score (C_score) provides a single, interpretable metric reflecting both the extent and intensity of corrosion present in the image.

The final output of the workflow includes side-by-side visualizations of the original image and the segmented mask, annotated with the corrosion severity score. This enables engineers to make quick, consistent, and data-driven decisions about asset condition and maintenance priorities. The approach provides significant improvement over traditional inspection methods, delivering a scalable, quantifiable, and repeatable process for corrosion assessment that can be deployed across large asset bases with minimal human intervention.

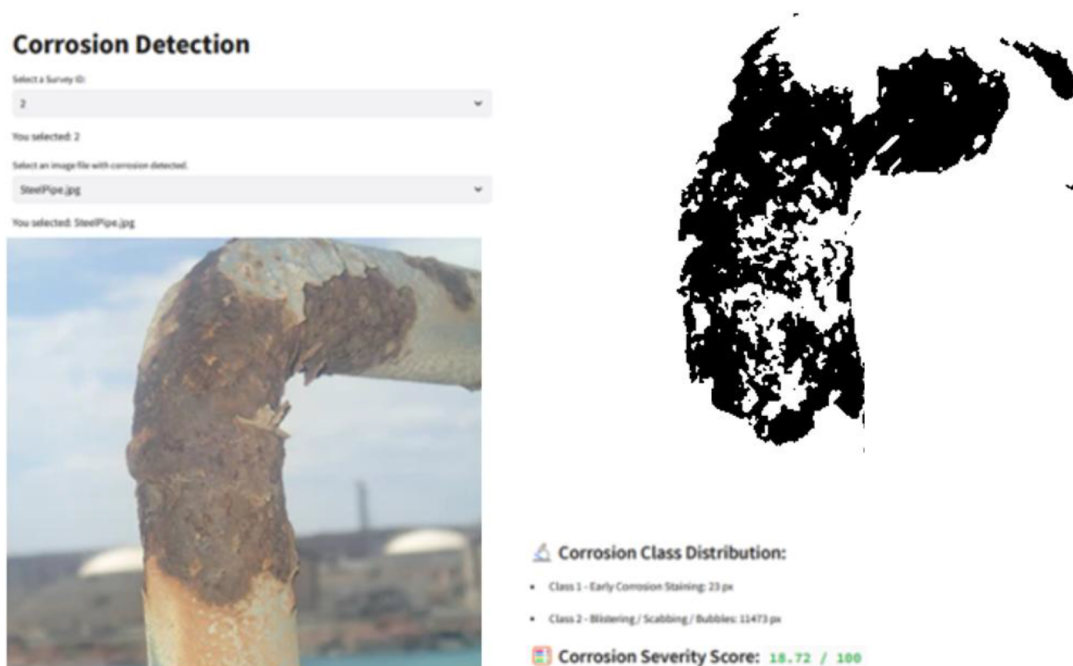


Figure 2—Corrosion Detection

Valve Detection and State Classification

Valves are critical mechanical elements within surface facility infrastructure, where their operational status, open or closed, directly impacts process safety, flow routing, and equipment isolation. Conventionally, valve state verification during inspections involves manual interpretation of lever or indicator positions captured through photographs or robotic imaging. This method is not only time-consuming but also susceptible to human error and inconsistency, particularly in large-scale surveys involving hundreds of components across multiple assets. To automate and standardize this process, a vision-based object detection model was developed to identify valves within images and classify their operational state.

Given the structured and localized nature of valve geometry, an object detection approach using the YOLO architecture was adopted. Unlike corrosion patterns that require pixel-level semantic segmentation, valves typically present well-defined contours, consistent spatial layouts, and are amenable to bounding-box-based detection. YOLO's architecture offered a favourable trade-off between accuracy and inference speed, with capability for real-time detection in field-deployable scenarios.

The training dataset was designed to capture a broad spectrum of visual characteristics associated with different valve types. It included both customer-provided inspection images and a diverse range of publicly available valve imagery. The full dataset consisted of 1,070 labeled images, divided into 1,000 training samples, 50 validation images, and 20 held-out test images. Images were manually annotated with bounding boxes and binary class labels: Class 0 for closed valves and Class 1 for open valves. These samples represented varying orientations, lever styles, lighting conditions, occlusions, and environmental contexts to maximize model robustness.

Training was performed using the YOLO detection framework over 50 epochs, with an input resolution of 640×640 pixels and a batch size of 16. Data augmentation techniques such as mosaic tiling, horizontal flipping, affine transformations, and color jitter were employed to reduce overfitting and enhance generalization across unseen field data. The model's architecture enabled simultaneous classification and bounding box regression, optimized through a compound loss function incorporating objectness, localization, and class confidence.

Valve Detection

Select a Survey ID:

2

You selected: 2

Select an image file with valve detected:

flange.jpg

You selected: flange.jpg



flange.jpg

Open (0.89)

Figure 3—Valve Detection

At inference, images are passed to the trained model, which returns a list of detected bounding boxes, each with a class prediction and a confidence score. A post-processing function was used to interpret class IDs into valve states. Specifically, Class 0 was mapped to "Closed" and Class 1 to "Open." Detected valves were rendered on the image with bounding boxes and color-coded labels, red for closed valves and green for open valves, with associated confidence scores. This allowed visual inspection teams to rapidly validate model outputs and identify discrepancies when necessary.

This system significantly reduced the manual workload associated with image-by-image valve state determination. It also enabled consistent interpretation across operators and surveys, with the added benefit of quantifiable confidence in each classification. Integration with the broader inspection platform ensured that valve state data could be stored, trended, and reviewed alongside other facility condition data, enabling automated anomaly tracking and decision support in future inspection cycles.

Gauge Reading via Vision-Based Inference

Analog gauges remain prevalent in offshore and brownfield assets, where direct sensor integration is limited by retrofit constraints or cost. These gauges, often pressure or temperature indicators, use a rotating mechanical needle that pivots from a central point and aligns with a marked scale. Conventional inspection involves image capture, manually or through drones/robotics, followed by human interpretation of the needle's position. This introduces latency, potential for misreading, and lack of traceability across inspection cycles. To resolve these challenges, a computer vision-driven method was developed to automatically infer the gauge value from image input, using bounding box detection of key reference points and a robust geometric interpretation pipeline.

The core solution centres on detecting four fixed geometric markers: the gauge center, minimum scale limit, maximum scale limit, and the needle tip. Unlike corrosion detection or other amorphous visual tasks that require pixel-level segmentation, the gauge reading problem lends itself naturally to discrete object detection. Each of these features has a consistent visual footprint and is non-overlapping in physical space, making them ideal candidates for bounding-box detection. YOLOv8 was chosen over YOLOv5 due

to improved precision in detecting small and tightly clustered visual features, which is critical for high-confidence needle and scale marker identification.

The training dataset consisted of 2,300 images, 1,000 for training, 1,000 for validation, and 300 for testing. Annotations followed a four-class schema corresponding to the reference points: centre, minimum, maximum, and tip. These were manually labeled across a variety of image conditions, including variations in lighting, glare, background clutter, and partial occlusions. The images were sourced from customer field data and public analog gauge datasets to ensure broad generalization across different gauge types and mounting orientations.

Once inference is executed, the model returns bounding boxes with class labels and confidence scores. From these, the highest-confidence box per class is selected. The geometric centers of each box are extracted and used as points in a 2D coordinate plane to establish the necessary vectors. Let:

- $C = (x_c, y_c)$ $C = (x_c, y_c)$ $C = (x_c, y_c)$: center of the gauge (pivot point)
- $T = (x_t, y_t)$ $T = (x_t, y_t)$ $T = (x_t, y_t)$: tip of the needle
- $M_{min} = (x_{min}, y_{min})$ $M_{\{min\}} = (x_{\{min\}}, y_{\{min\}})$ $M_{min} = (x_{min}, y_{min})$ reference point
- $M_{max} = (x_{max}, y_{max})$ $M_{\{max\}} = (x_{\{max\}}, y_{\{max\}})$ $M_{max} = (x_{max}, y_{max})$ reference point

The absolute angle of a vector is computed using the 2D arctangent function:

$$\theta = \text{atan2}(y_{target} - y_c, x_{target} - x_c)$$

normalize this into the 0deg to 360deg range for clockwise measurement, the angle is adjusted as:

$$\theta_c = \begin{cases} \theta \times \frac{180}{\pi} + 360, & \text{if } \theta < 0 \\ \theta \times \frac{180}{\pi}, & \text{otherwise} \end{cases}$$

Using this, angles for minimum, maximum, and tip are calculated as θ_{min} , θ_{max} , θ_{tip} respectively. The needle's effective displacement from minimum is computed as:

$$\theta_{needle} = (\theta_{tip} - \theta_{min}) \bmod 360\text{deg}$$

This ensures that the measured arc always represents clockwise rotation, irrespective of quadrant orientation or scale type (e.g., 270° gauges).

To convert this angle into a real-world measurement, the algorithm uses linear proportionality:

$$\text{Raw Value} = (\theta_{needle} / \theta_{max} - \theta_{min}) \times (\text{Max Scale} - \text{Min Scale})$$

To address minor inconsistencies introduced by image skew, lens distortion, or gauge misalignment, a correction factor k is applied:

$$\text{Final Value} = \text{Raw Value} \times k$$

In this implementation, the value of $k=0.27$ was calibrated using field-verified measurements across multiple assets. This factor serves to align predicted gauge readings with ground-truth instrument outputs and corrects for common optical distortions.

The pipeline concludes by rendering visual feedback: bounding boxes, connecting lines, angles, and predicted measurements are drawn over the image. The output is saved and optionally published to a cloud interface for engineering verification, audit logging, or integration into downstream analytics.

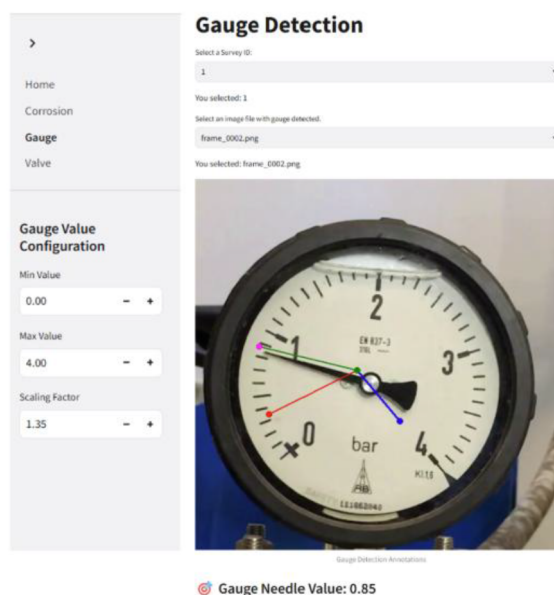


Figure 4—Gauge Detection

This approach offers robust, scalable, and interpretable digitization of analog gauge readings, transforming previously manual image reviews into a structured, repeatable, and fully auditable process. It supports a wide range of inspection formats, from routine field surveillance to ad-hoc photographic submissions, and enables integration with broader asset health monitoring frameworks.

Results and Performance Evaluation

The developed system was evaluated both quantitatively and qualitatively across multiple surface inspection use cases, including corrosion classification, valve state detection, and analog gauge reading. Each model was benchmarked on test datasets and deployed in real-world operational environments to validate scalability, robustness, and usability.

Model Accuracy and Generalization

Corrosion Detection. The corrosion model, implemented using ResNet34-based U-Net architecture, was trained on 1,000 annotated images using a 90/10 split. It achieved a mean Intersection over Union (mIoU) of 0.81 and pixel-wise classification accuracy of 88.3% on the test set. The model successfully segmented four corrosion categories, no corrosion, early staining, blistering/scabbing, and flaking, and enabled quantitative evaluation through a computed Corrosion Severity Score. This score was calculated using pixel area weighted by corrosion class severity, normalized to a 0–100 scale, and provided a consistent index of degradation to support maintenance prioritization.

Valve Detection. The YOLO-based model was trained on 1,070 images (1,000 train, 20 test, 50 validation), detecting "open" and "closed" valve states. Evaluation yielded a mean Average Precision (mAP@0.5) of 92.6%. The system rendered bounding boxes directly on the image, with green boxes indicating open valves and red for closed, along with confidence scores, enabling easy field verification.

Gauge Detection. A YOLOv11 model was trained on 2,300 images (1,000 train, 300 test, 1,000 validation) to identify four-gauge components: needle tip, center, minimum, and maximum. It achieved class-wise detection accuracies above 90%. Post-processing calculated the angle between the center and needle tip with respect to the minimum mark using the formula:

$$\theta = \text{atan2}(y_{\text{tip}} - y_c, x_{\text{tip}} - x_c)$$

The measured angle was normalized to 0–360° and converted to a real-world value via:

$$Reading = (\theta_{needle} - \theta_{min} \theta_{max} - \theta_{min}) \times (Max - Min) \times Correction\ Factor$$

A correction factor of 0.27(configurable) was used based on field calibration. Overall, the gauge reading estimates demonstrated less than ±5% deviation when compared with manual values.

Operational Efficiency and Latency Reduction

To handle the large image volumes typically associated with robotic surveys, the solution incorporated multithreaded execution, allowing up to five concurrent detection functions. This reduced per-image inference latency by 35–40% compared to sequential execution. The pipeline architecture allowed independent processing of corrosion, valve, and gauge modules with synchronized output generation.

Detection outputs were stored as structured.csv files (e.g., Image123_CD.csv) and tagged with unique metadata fields including CD_ID, VD_ID, GD_ID, and SURVEY_ID. These tags enabled downstream traceability and auditability. Metadata also included status fields (OK, FAIL, ND) to signal inference health per module.

Detection Metadata Fields and Descriptions

Table 1—Metadata Description

Metadata Key	Description
CD	Status of Corrosion Detection (DONE, ND, FAIL)
CD_ID	Unique identifier for the corrosion detection result CSV
VD	Status of Valve Detection (DONE, ND, FAIL)
VD_ID	Unique identifier for the valve detection result CSV
GD	Status of Gauge Detection (DONE, ND, FAIL)
GD_ID	Unique identifier for the gauge detection result CSV
STATUS	Composite status summarizing detection outcomes (e.g., CD-OK, VD-FAIL)
SURVEY_ID	Unique identifier assigned to the survey during image upload or transformation

Real-World Validation and Operator Feedback

The system was tested in production-like environments under collaboration with two major North Sea operators with global operations. The first test environment used archived inspection data featuring diverse corrosion and equipment layouts, while the second used open-source datasets representing general facility infrastructure.

In both cases, the system demonstrated strong detection capability across varying image conditions and inspection scenarios. Users were able to upload images via the UI, run inference workflows, and filter results by SURVEY_ID. Corrosion overlays, valve bounding boxes, and dynamic gauge reading calculations were highlighted as key differentiators compared to traditional image review workflows. In future this workflow can become part of an agentic system, where user can trigger it with a simple prompt to Agent or Large Language Model interface [5].

References

1. Cognite. 2023. Data Contextualization for Industry. White Paper. https://6407318.fs1.hubspotusercontent-na1.net/hubfs/6407318/2023-White_Paper-Contextualization.pdf
2. Shekhawat, D.S., Wright, M., Bell, A., Barua, J., Bhatia, K., and Wood, B. 2025. AI-Driven Robotics for Efficient Subsea Pipeline Inspections in the North Sea. Presented at the EAGE Conference, 1–5. <https://www.earthdoc.org/content/papers/10.3997/2214-4609.2025101474>

3. Terven, J., Córdova-Esparza, D.-M., and Romero-González, J.-A. 2023. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction* **5** (4): 1680–1716. <https://doi.org/10.3390/make5040083>
4. Ronneberger, O., Fischer, P., and Brox, T. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, *Lecture Notes in Computer Science*, Vol. **9351**, 234–241. Springer, Cham. https://doi.org/10.1007/978-3-319-24574-4_28
5. Shekhawat, D., Barua, J., Bhatia, K., and Saumya, S. 2025. Offshore Production Surveillance and Intervention Using Multi-Agent AI. Paper SPE-226728-MS presented at the SPE Offshore Europe Conference & Exhibition, Aberdeen, UK, 2-5 September.